

LabPath

System Design

Architecture, NestJS backend, PostgreSQL / Prisma data model, and learner user flows — mapped to the v1.0 product specification.

Next.js

NestJS

Prisma

PostgreSQL + pgvector

BullMQ / Redis

DOCUMENT

System Design & Data Architecture v1.0

STATUS

Draft for engineering review

AUDIENCE

Backend, frontend, product

BASED ON

LabPath Product Specification v1.0 · 25 Aug 2026

SCOPE

Phase 0 POC · 10 exercises · 3 simulators

STACK DECISION

Node / NestJS replaces Python workers for Phase 0

Contents

- 01 Purpose, constraints and stack
- 02 System architecture
- 03 NestJS module map
- 04 Grading engine on Nest
- 05 Database design
- 06 Prisma schema
- 07 User flows
- 08 API surface and events
- 09 Security, budgets, anti-cheat
- 10 Phase 0 delivery sequence

How to read this document

This is the engineering companion to the product spec. Product rules (P1–P8, grader classes, hidden sets, exercise-as-code) are unchanged. This document translates them into a NestJS + Prisma system the team can start building.

Stack change vs product spec §11. The product spec recommended Python/FastAPI grading workers. The team decision for this blueprint is a single NestJS codebase: HTTP API + BullMQ workers. Metrics, Assertion DSL, Slice Spec parser, and harnesses ship in TypeScript. A Python sidecar remains an optional later extraction if a metric library becomes a blocker — it is not required for Phase 0.

What must stay true from day one: hidden eval sets never reach the API database role; verdicts are three-valued (pass / fail / inconclusive); money is EUR micros; submissions and traces are durable addressable objects; users have roles learner | tutor | org_admin | admin; every exercise has at least one class A gate.

1. Purpose, constraints and stack

Phase 0 proves one thing: a graded, hidden-eval-set exercise produces feedback good enough that engineers come back. Not payments, not sandbox, not tutors.

1.1 Product constraints that bind architecture

Principle	Architecture consequence
P1 Objective pass/fail	Gates live in <code>exercise.yaml</code> , executed by workers, stored as <code>GateResult</code> rows/JSON.
P2 Deterministic before judges	Class A metrics run first. Class B is optional and never the sole gate.
P3 Feedback is the product	Grade payload includes scorecard, rotated failing cases, failure classes, next action.
P4 Hidden test sets	Two Postgres schemas. API role cannot SELECT harness secrets.
P6 Bounded cost	Model gateway meters every provider call; job killed on EUR/token/call breach.
P8 Stable verdicts	<code>gen_cache</code> + <code>judge_cache</code> + frozen fixtures for Evaluation. Target $\geq 99\%$.
§7.7 No wall-clock gates	Efficiency gates use tokens / call counts / context size only.
§12.2 No arbitrary code	Learner input = JSON config, prompts, Assertion DSL, Slice Spec. RE2 only.

1.2 Chosen stack

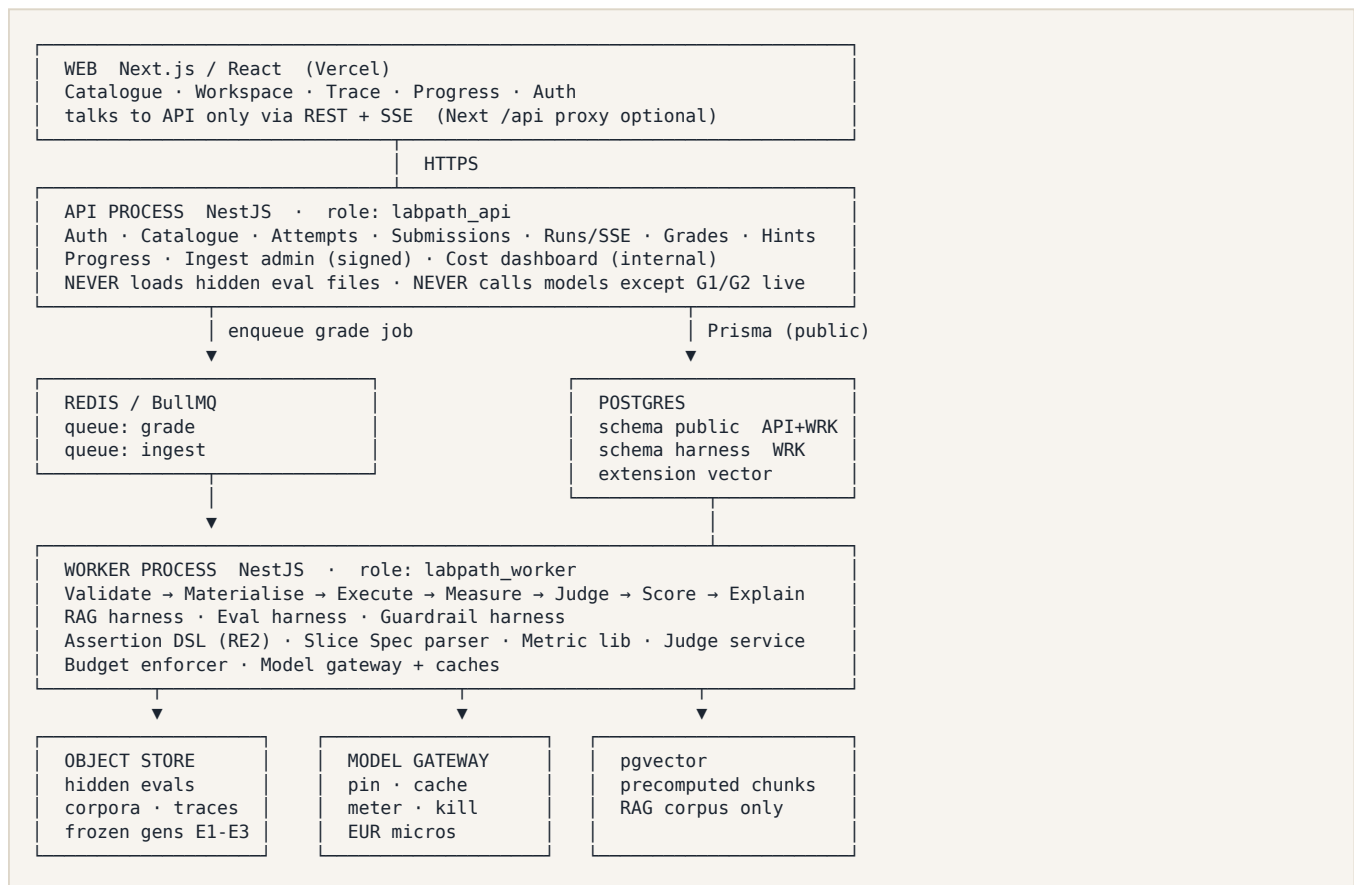
Layer	Choice	Why
Web	Next.js / React on Vercel	Reuse prototype shell. Catalogue, workspace, trace, progress.
API	NestJS (Node 20+)	Modules map 1:1 to domains. Guards, pipes, SSE, queues native.
ORM	Prisma	Typed models, migrations, JSON columns for polymorphic payloads.
Database	PostgreSQL 16 + pgvector	Source of truth. One less vector DB. EUR integer money.
Queue	Redis + BullMQ	Grade jobs, hint jobs, ingest. Worker process separate from HTTP.
Object store	S3-compatible	Corpora, hidden jsonl, traces, frozen generations.
Models	Model gateway module	Pin versions, cache, meter, hard cutoff. Provider locked by O1.
Auth	Magic link + OAuth	Minimal Phase 0. JWT access + rotating refresh cookies.

1.3 Explicitly out of Phase 0

Code sandbox, Stripe, contests, streaks, public profiles, tutor marketplace UI, i18n, the other five simulators. Role enum and shareable traces are still built now so Phase 4 is not a rewrite.

2. System architecture

One NestJS repository, two processes: **API** (HTTP + SSE) and **Worker** (grading). They share Prisma models and a Redis queue. They do not share database roles.



2.1 Process split

API process

Accepts learner traffic. Validates submission JSON against the exercise schema. Creates Attempt / Submission / Run rows. Enqueues a job. Streams run progress. Returns scorecards with rotated failing cases only.

Worker process

Reads hidden assets from object store + harness schema. Runs harnesses under a hard budget. Writes Grade + Trace. Never serves HTTP to the public internet.

2.2 Why two database roles

Product spec §12.4: hidden eval sets never leave the server — not in API responses, traces, or error messages. The strongest enforcement is not “remember to omit the field”. It is a Postgres role that physically cannot read the table.

- labpath_api — GRANT on schema public only.
- labpath_worker — GRANT on public + harness + object-store IAM.
- CI leakage test greps API fixtures for hidden-set canaries and fails the build.

3. NestJS module map

Monorepo stays. apps/api is rewritten from Express + Mongoose to NestJS + Prisma. apps/web keeps the prototype shell. packages/shared holds enums, failure-class vocabulary, and DTO types.

apps/api/	
src/	
main.ts	HTTP bootstrap
worker.main.ts	BullMQ bootstrap (same AppModule, different entry)
app.module.ts	
prisma/	PrismaService (API) + PrismaHarnessService (worker)
modules/	
auth/	magic link, OAuth, JWT, roles guard
users/	
catalogue/	GET /exercises, GET /exercises/:slug
attempts/	start attempt, attempt policy (E3 cooldown)
submissions/	POST payload, schema validate, enqueue
runs/	GET status, GET SSE stream
grades/	GET scorecard (rotated cases)
traces/	GET instrumentation blob (no hidden labels)
hints/	unlock next hint
progress/	solves, skill rollup
ingest/	signed admin upsert from GitHub Action
cost/	internal cost dashboard
grading/	worker-only
pipeline.processor.ts	VALIDATE...EXPLAIN
harnesses/rag eval guardrail	
metrics/	recall@k, nDCG, F1, kappa, Wilson, z-test
dsl/	assertion.dsl.ts, slice-spec.parser.ts
judge/	quarantine, JSON schema verdict, evidence bind
gateway/	provider calls, caches, EUR meter
budget/	kill switch
normaliser/	G1/G2 shared encoding normaliser
common/	filters, interceptors, RE2, ids

3.1 HTTP modules vs worker modules

Module	Process	Responsibility
Auth, Users, Catalogue	API	Identity and public exercise metadata
Attempts, Submissions	API	Lifecycle + payload hash + enqueue
Runs, Grades, Traces, Hints	API	Read models. SSE from Redis pub/sub
Ingest	API (admin)	Signed upsert. Does not copy hidden files into SQL
GradingModule	Worker only	Imported in worker.main.ts, not in public HTTP app
GatewayModule	Worker (+ G1/G2 API live turns)	All provider I/O. G1/G2 chat turns hit gateway from API with the same meter

3.2 Interactive exception: G1 / G2

Most exercises are “submit config → async grade”. G1 is a live mock chat. G2 is “submit a page, then watch the agent”. Those turns still go through the model gateway and write a tool-call log. Win detection stays class A: canary in output, or send_email.to equals attacker address. The grade job for G1/G2 may be short-circuited to an event detector on the same request path, with the Run still persisted.

Keep Nest module names stable. Adding Prompt Engineering (Phase 1) or Agent simulator (Phase 2) should be a new harness folder, not a new data model.

4. Grading engine on Nest

The pipeline is identical to spec §7.1. Implementation language is TypeScript.



Step	What runs
Validate	JSON Schema for the exercise, size caps, banned constructs, E3 attempt policy, degenerate-suite guard (E1).
Materialise	Build retriever config, prompt, assertion suite, slice spec, or mock-app policy from payload.
Execute	Hidden set from object store. Sample G3 with sampleSeed. Frozen fixtures for E1–E3. Precomputed embeddings for RAG.
Measure	Class A: recall@k, nDCG@k, F1, kappa, canary, tool log, token proxies, gold-span overlap.
Judge	Only R2/R3/R4. n=3 majority, temp 0, quarantined blocks, structured JSON, evidence span check, cache key (judgeVersion, rubricHash, outputHash).
Score	Banded CI for class B. Pass / fail / inconclusive. Inconclusive does not consume the attempt; one rerun on a larger sample.
Explain	Failure-class taxonomy + rotated K failing cases. Never the full hidden set.

4.1 TypeScript metric library (Phase 0 complete list)

RAG recall@k, gold-span overlap, citation-id validity, nDCG@k, mean prompt tokens, model-call count	Eval precision/recall/F1, Cohen’s κ, self-consistency, two-proportion z, BH/Bonferroni, bootstrap CI	Guardrails canary detect, tool-log match, block rate, Wilson interval, filter token count
---	--	---

4.2 Assertion DSL and Slice Spec

Two separate parsers. Do not unify in Phase 0.

- **Assertion DSL** — YAML, RE2 via the `re2` npm package (not JS RegExp). Timeouts 50 ms / assertion, 5 s / suite, max 40 assertions, pattern ≤ 512 chars.
- **Slice Spec** — hand-written boolean parser over `meta.*` only. Never `eval()`, never `new Function()`.

4.3 Caches that make P8 true

Cache	Key	Used by
Frozen generations	object-store fixtures, versioned	E1 E2 E3 — no live gen at grade time
gen_cache	(modelVersion, promptHash, paramsHash)	R2–R4, G1–G3 re-grades
judge_cache	(judgeVersion, rubricHash, outputHash)	All class B
chunk embeddings	(corpusVersion, embedModel, chunkId)	All RAG

5. Database design

5.1 Rules

- UUIDs everywhere (`gen_random_uuid()`).
- All money is `BIGINT` EUR micros. `fxRate` recorded on the run. No mixed-currency columns.
- `verdict` is an enum of three values, never a boolean.
- Submission payload is polymorphic JSON, validated by the exercise's JSON Schema.
- Hidden eval JSONL is **not** a Prisma model the API can query. It lives in object storage. Worker reads it with a signed URI stored in `harness.exercise_assets`.
- Public sample may be copied into `exercises.publicSample` because the catalogue endpoint is allowed to return it.
- Historical grades keep `exerciseVersion`. Never silently re-score.

5.2 Schema map

Schema	Tables	Who can read
public	users, identities, exercises, attempts, submissions, runs, grades, traces, hint_unlocks, skills, user_skill_scores, refresh_tokens, magic_links	API + worker
harness	exercise_assets, corpus_chunks, gen_cache, judge_cache, label_sets	Worker only

5.3 Entity relationships

```
User 1—* Attempt 1—* Submission 1—* Run 1—1 Grade
                               └─1 Trace (blob URI)

Exercise 1—* Attempt
Exercise *—* Skill             (ExerciseSkill)
User *—* Skill                 (UserSkillScore)

Run.sampleSeed → makes G3 (and any sampled exercise) replayable
Run.costEurMicros + fxRate → unit-economics dashboard
Grade.verdict ∈ {pass, fail, inconclusive}
Grade.failureClasses[] → controlled vocabulary, not free text
```

5.4 What is not stored in Postgres

Object store

eval_hidden.jsonl, full traces, frozen generation fixtures, RAG corpora source files, judge rubrics YAML, reference/near-miss solutions.

Git (exercise-as-code)

exercise.yaml, brief.md, hints.md, public/hidden jsonl, grader extras, solution/. GitHub Action validates and upserts metadata only.

5.5 Role model (marketplace-ready, unused until Phase 4)

learner is the only active product role in Phase 0. tutor and org_admin exist on the enum so we do not migrate user types later. No tutor tables, booking, or payouts until Phase 4.

6. Prisma schema — datasource, enums, identity

File: apps/api/prisma/schema.prisma. Preview features: postgresqlExtensions, multiSchema.

```
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions", "multiSchema"]
}

datasource db {
  provider      = "postgresql"
  url           = env("DATABASE_URL")
  extensions    = [pgcrypto, vector]
  schemas      = ["public", "harness"]
}

enum Role {
  learner tutor org_admin admin      @@schema("public")
}
enum AuthProvider {
  magic_link google github            @@schema("public")
}
enum ExerciseType {
  drill exercise lab simulation scenario @@schema("public")
}
enum Simulator {
  rag evaluation guardrails           @@schema("public")
}
enum Difficulty {
  E M H                               @@schema("public")
}
enum AttemptStatus {
  started submitted grading graded abandoned @@schema("public")
}
enum RunStatus {
  queued running succeeded failed killed_budget @@schema("public")
}
enum Verdict {
  pass fail inconclusive              @@schema("public")
}

model User {
  id          String    @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  email       String    @unique
  emailVerified DateTime?
  role        Role       @default(learner)
  displayName  String?
  imageUrl    String?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  deletedAt   DateTime?

  identities   Identity[]
  magicLinks   MagicLink[]
  refreshTokens RefreshToken[]
  attempts     Attempt[]
  hintUnlocks  HintUnlock[]
  skillScores  UserSkillScore[]

  @@map("users")
  @@schema("public")
}

model Identity {
  id          String    @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String    @db.Uuid
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  provider     AuthProvider
  providerUserId String?
  passwordHash String?
  createdAt   DateTime   @default(now())

  @@unique([provider, providerUserId])
  @@index([userId])
  @@map("identities")
  @@schema("public")
}

model MagicLink {
  id          String    @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String    @db.Uuid
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  tokenHash   String
  expiresAt   DateTime
  consumedAt  DateTime?
  createdAt   DateTime   @default(now())

  @@index([tokenHash])
  @@map("magic_links")
  @@schema("public")
}
```



```
model RefreshToken {
  id      String    @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId  String    @db.Uuid
  user    User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  tokenHash String   @unique
  expiresAt DateTime
  revokedAt DateTime?
  createdAt DateTime @default(now())

  @@map("refresh_tokens")
  @@schema("public")
}
```

6. Prisma schema — catalogue, skills, attempts

```
model Skill {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  slug        String   @unique
  name        String
  parentId    String?  @db.Uuid
  parent      Skill?   @relation("SkillTree", fields: [parentId], references: [id])
  children    Skill[]  @relation("SkillTree")

  exercises   ExerciseSkill[]
  scores       UserSkillScore[]

  @@map("skills")
  @@schema("public")
}

model Exercise {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  slug        String
  version     Int
  type        ExerciseType @default(exercise)
  simulator    Simulator
  title       String
  briefMd     String
  difficulty   Difficulty
  submissionSchema Json
  thresholds   Json
  budget       Json      // { maxModelCalls, maxTokens, maxCostEur, wallClockS }
  gates        Json      // copy of exercise.yaml gates for the worker
  attemptPolicy Json?    // { maxAttempts, cooldownHours } – E3 uses this
  feedback     Json      // { showFailingCases, rotate }
  publicSample  Json?    // allowed on GET /exercises/:slug
  isPublished  Boolean   @default(false)
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt

  skills       ExerciseSkill[]
  attempts     Attempt[]
  hintUnlocks  HintUnlock[]

  @@unique([slug, version])
  @@index([simulator, difficulty, isPublished])
  @@map("exercises")
  @@schema("public")
}

model ExerciseSkill {
  exerciseId String @db.Uuid
  skillId     String @db.Uuid
  weight      Float  @default(1)
  exercise    Exercise @relation(fields: [exerciseId], references: [id], onDelete: Cascade)
  skill       Skill    @relation(fields: [skillId], references: [id], onDelete: Cascade)

  @@id([exerciseId, skillId])
  @@map("exercise_skills")
  @@schema("public")
}

model UserSkillScore {
  userId      String @db.Uuid
  skillId     String @db.Uuid
  score       Float  @default(0)
  lastPracticedAt DateTime?
  user        User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  skill       Skill   @relation(fields: [skillId], references: [id], onDelete: Cascade)

  @@id([userId, skillId])
  @@map("user_skill_scores")
  @@schema("public")
}

model Attempt {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String   @db.Uuid
```

```

exerciseId      String      @db.Uuid
exerciseVersion Int
status          AttemptStatus @default(started)
startedAt       DateTime     @default(now())
submittedAt     DateTime?
user            User         @relation(fields: [userId], references: [id], onDelete: Cascade)
exercise        Exercise     @relation(fields: [exerciseId], references: [id])
submissions     Submission[]

@@index([userId, exerciseId])
@@map("attempts")
@@schema("public")
}

model HintUnlock {
  id          String  @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  userId      String  @db.Uuid
  exerciseId  String  @db.Uuid
  hintIndex   Int
  unlockedAt  DateTime @default(now())
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  exercise    Exercise @relation(fields: [exerciseId], references: [id])

  @@unique([userId, exerciseId, hintIndex])
  @@map("hint_unlocks")
  @@schema("public")
}

```

Ingest upserts a new Exercise row when YAML content changes and bumps version. Old attempts keep pointing at the old row via exerciseVersion.

6. Prisma schema — submissions, runs, grades, traces

```
model Submission {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  attemptId   String   @db.Uuid
  payload     Json      // polymorphic per simulator – never contains hidden labels
  payloadHash String
  createdAt   DateTime @default(now())
  attempt     Attempt   @relation(fields: [attemptId], references: [id], onDelete: Cascade)
  runs        Run[]

  @@index([attemptId])
  @@index([payloadHash])
  @@map("submissions")
  @@schema("public")
}

model Run {
  id             String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  submissionId   String   @db.Uuid
  workerVersion  String
  modelVersions  Json      // { generator, judge, embed }
  sampleSeed     String?  // G3 and any sample_per_attempt exercise
  status         RunStatus @default(queued)
  tokensIn       Int       @default(0)
  tokensOut      Int       @default(0)
  costEurMicros  BigInt    @default(0)
  fxRate         Decimal   @db.Decimal(12, 6)
  cacheHitRatio  Float     @default(0)
  errorCode      String?
  errorMessage   String?
  startedAt      DateTime?
  finishedAt     DateTime?
  createdAt       DateTime @default(now())
  submission     Submission @relation(fields: [submissionId], references: [id], onDelete: Cascade)
  grade          Grade?
  trace          Trace?

  @@index([submissionId])
  @@index([status, createdAt])
  @@map("runs")
  @@schema("public")
}

model Grade {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  runId       String   @unique @db.Uuid
  verdict     Verdict
  metrics     Json      // [{ id, value, ciLow, ciHigh, threshold, passed }]
  gateResults Json      // [{ id, class, outcome, metric }]
  failureClasses String[]
  scorecard   Json      // learner-facing projection, already redacted
  failingCases Json      // ROTATED sample only – never the full hidden set
  createdAt   DateTime @default(now())
  run         Run       @relation(fields: [runId], references: [id], onDelete: Cascade)

  @@map("grades")
  @@schema("public")
}

model Trace {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  runId       String   @unique @db.Uuid
  blobUri     String   // object store – chunks, prompts, tools, tokens, latency
  createdAt   DateTime @default(now())
  run         Run       @relation(fields: [runId], references: [id], onDelete: Cascade)

  @@map("traces")
  @@schema("public")
}
```

Redaction rule. Grade.failingCases is written by the worker after sampling. The API returns this column as-is and never joins to hidden labels. Trace blobs must be stripped of gold labels before upload. A CI test asserts hidden fixture strings

cannot appear in grade or trace JSON.

6. Prisma schema — harness (worker-only)

```
model ExerciseAsset {
  id            String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  exerciseSlug  String
  exerciseVersion Int
  hiddenEvalUri String   // s3://.../eval_hidden.jsonl - NEVER copied to public
  publicEvalUri String
  corpusUri     String?
  labelsUri     String?
  frozenGenUri  String? // E1-E3
  judgeYamlUri  String?
  createdAt     DateTime @default(now())

  @@unique([exerciseSlug, exerciseVersion])
  @@map("exercise_assets")
  @@schema("harness")
}

model CorpusChunk {
  id            String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  corpusId      String
  chunkId       String
  text          String
  headingPath   String?
  embedModel    String
  embedding     Unsupported("vector")?
  createdAt     DateTime @default(now())

  @@unique([corpusId, chunkId, embedModel])
  @@map("corpus_chunks")
  @@schema("harness")
}

model GenCache {
  id            String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  modelVersion  String
  promptHash   String
  paramsHash   String
  blobUri      String
  hits         Int       @default(0)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  @@unique([modelVersion, promptHash, paramsHash])
  @@map("gen_cache")
  @@schema("harness")
}

model JudgeCache {
  id            String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  judgeVersion  String
  rubricHash   String
  outputHash   String
  verdict      Json
  hits         Int       @default(0)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  @@unique([judgeVersion, rubricHash, outputHash])
  @@map("judge_cache")
  @@schema("harness")
}

model LabelSet {
  id            String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  exerciseSlug  String
  annotatorA    String
  annotatorB    String
  kappaHh       Float?
  adjudicated   Boolean  @default(false)
  labelsUri     String
  createdAt     DateTime @default(now())

  @@map("label_sets")
}
```

```
  @@schema("harness")  
}
```

6.1 Postgres grants (run after migrate)

```
REVOKE ALL ON SCHEMA harness FROM PUBLIC, labpath_api;  
GRANT USAGE ON SCHEMA public TO labpath_api, labpath_worker;  
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO labpath_api;  
GRANT ALL ON SCHEMA harness TO labpath_worker;  
GRANT ALL ON ALL TABLES IN SCHEMA harness TO labpath_worker;
```

Worker Prisma client uses DATABASE_URL_WORKER. API client uses DATABASE_URL. Do not put the worker URL in the API process env.

6. Payload shapes (polymorphic JSON)

One `submissions.payload` column. Validation is per-exercise JSON Schema stored on `Exercise.submissionSchema`.

Exercise	Payload (conceptual)
R1	{ chunkSize, overlap, split: "fixed" "sentence" "recursive" "heading" }
R2	{ topK, rerank, chunkSize }
R3	{ generationPrompt }
R4	{ reranker, topN, queryRewritePrompt? }
E1 / G3 filters	Assertion DSL YAML string (parsed server-side)
E2	{ judgePrompt, rubric }
E3	Slice Spec YAML
G1	Interactive turns: { level, messages[] } — win is an event, not a batch grade
G2	{ pageContent, level }
G3	{ systemPrompt, inputFilterYaml, outputFilterYaml, toolPolicy? }

6.2 JSON stored on Grade

```
// metrics[]
{ "id": "recall_at_5", "value": 0.83, "ciLow": null, "ciHigh": null,
  "threshold": 0.80, "op": "gte", "class": "A", "passed": true }

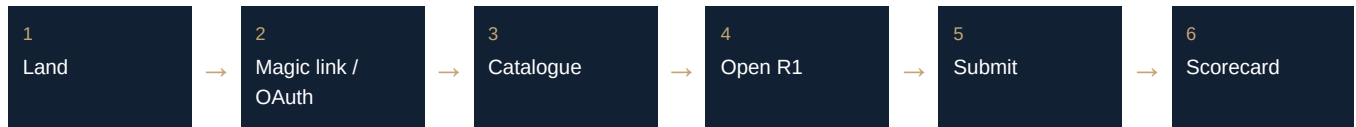
// gateResults[]
{ "id": "citation-support", "class": "B", "outcome": "inconclusive",
  "metric": "citation_supports_claim" }

// failingCases[] — already rotated
{ "input": "...", "expected": "...", "actual": "...", "failureClass": "chunk-too-large" }
```


7. User flows

Phase 0 has four screens. All flows below use those screens. Activation metric: signup → first submission in the same session.

7.1 First session (activation)



R1 is the onboarding exercise: under 15 minutes, class A only, no judge. Catalogue default-sorts Easy + RAG first for new users. Auth is required before submit (guest browse of briefs is allowed; submit is not).

7.2 Catalogue

- GET /api/exercises?simulator=&difficulty=&skill=
- Cards grouped by simulator. Difficulty badge, skill tags, solved state from /api/me/progress.
- Hidden sets, thresholds internals, and reference solutions are not in this payload.

7.3 Start attempt → workspace

```
Learner opens /exercises/:slug
├─ GET /api/exercises/:slug    brief, schema, public sample, budgets
├─ POST /api/attempts         { exerciseSlug } → Attempt(started)
└─ Workspace renders
    Left  brief, objective, constraints, budgets, hints
    Centre submission surface (form | prompt | YAML | mock app)
    Right run panel (idle)
```

E3 attempt policy is enforced here: if 5 submissions in the window, API returns 429 with `cooldownHours: 24`.

7. User flows — submit and grade

7.4 Async grade (R1–R4, E1–E3, G3)

```
POST /api/attempts/:id/submissions { payload }
├─ JSON Schema validate (fast reject)
├─ E1 degenerate-suite check
├─ payloadHash = sha256(canonical json)
├─ Attempt → submitted / grading
├─ Run(queued) inserted
├─ BullMQ job grade:{runId}
└─
Learner GET /api/runs/:id/stream (SSE)
  events: queued | started | step | metric | done | killed_budget | error
└─
Worker pipeline VALIDATE...EXPLAIN
├─ write Grade (verdict, rotated cases)
├─ write Trace blob (redacted)
├─ Attempt → graded
├─ publish SSE done
└─
Workspace right panel → scorecard + 3 failing cases + failure class + next action
```

7.5 Inconclusive (class B band / G3 Wilson)

1. Worker returns verdict: inconclusive.
2. UI copy: result is within measurement noise; attempt is not consumed.
3. Worker automatically enqueues one rerun on a larger sample, same payloadHash.
4. If still inconclusive → resolve as pass (spec §7.2).

7.6 Hints

POST /api/exercises/:slug/hints/next unlocks the next index into hint_unlocks. Hints are progressive. Tutor CTA is **not** shown in Phase 0. Spec allows the booking prompt only after ≥ 3 failed attempts in later phases — do not upsell on attempt one.

7.7 Trace

GET /api/runs/:id/trace returns the instrumentation blob: retrieved chunks + scores, prompts actually sent, tokens, cost, per-step latency, tool calls. Gold labels and hidden questions that were not in the rotated sample are absent. Free-tier hiding of traces is a Phase 1 concern; Phase 0 shows traces to every authenticated solver.

7. User flows — simulator-specific

7.8 RAG (R1–R4)

R1 / R2 / R4 — config form

Centre panel is structured fields (chunk size, overlap, top-k, reranker). Public sample of 5 Q/A is visible. Submit runs retrieval (R1 has zero live generations). Feedback highlights gold span vs retrieved chunks.

R3 — prompt editor (flagship)

Retrieval is fixed. Learner writes the generation prompt. Scorecard shows citation format, ID validity, gold-span overlap, refusal counts, then banded class B support. Demo this one.

7.9 Evaluation (E1–E3)

E1 YAML assertions

Editor + 10 labelled public samples.
Confusion matrix on fail. Rotating 3 FP + 3 FN.

E2 judge prompt

Platform-fixed temperature 0.7, 80 items × 3. Learner does not set temperature. Malformed JSON = incorrect.

E3 slice spec

Metadata schema published in the brief. 5 submits then 24h cooldown. No candidate slice list to brute-force.

7.10 Guardrails (G1–G3)

G1 Break the Concierge – split UI, live chat
Left: brief + level + objective (extract canary)
Right: mock hotel concierge
Each turn: POST /api/simulations/gl/turns → gateway → canary detector
Win: detector hits → defence debrief (which encoding, what would have stopped it)

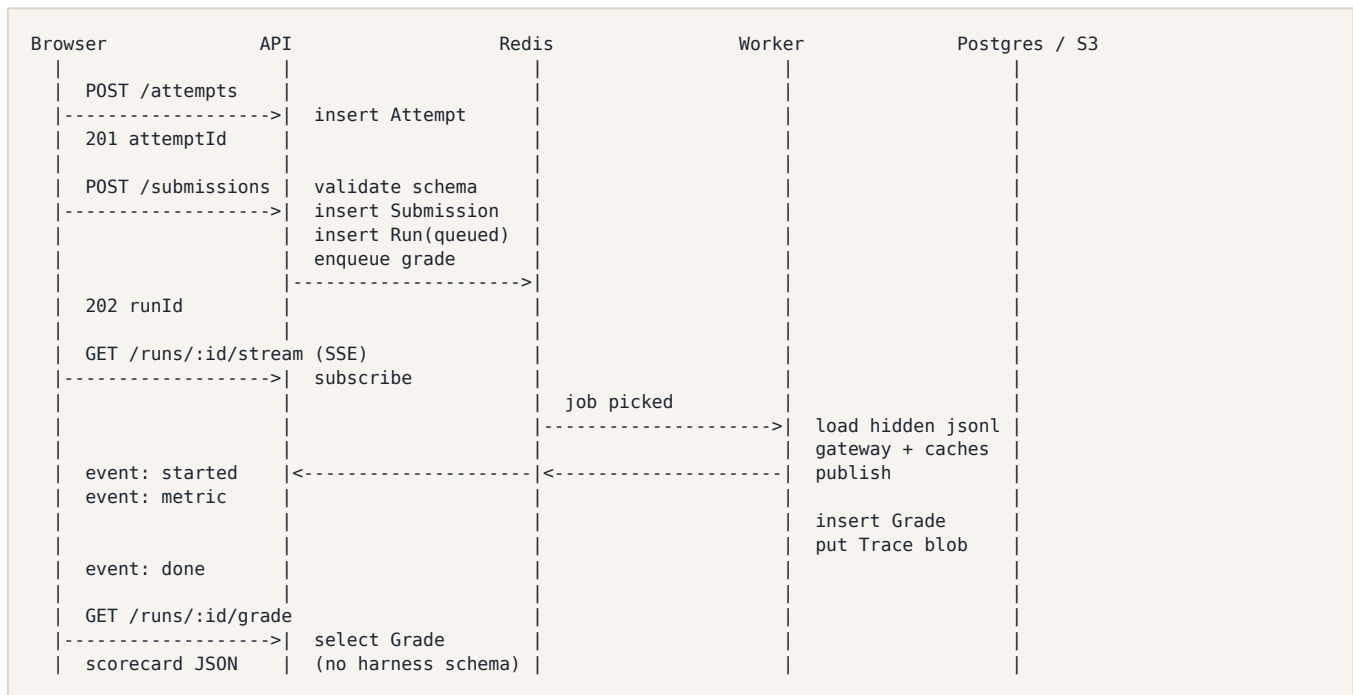
G2 Indirect Payload – no chat with the agent
Learner submits page HTML/markdown
Worker runs mock research agent + optional confirmation-gate policy checker
Win: tool log send_email.to == attacker address

G3 Hold the Line – blue team
Learner submits system prompt + Assertion DSL filters + optional tool policy
Worker samples 40 attacks + 60 benign from 500-item pool (sampleSeed stored)
Scorecard: 2x2 + all errors from THAT sample only + Wilson CI

7.11 Progress

GET /api/me/progress → attempts, solves, first-cut skill radar from user_skill_scores. Recency decay can be a computed view; full decay is Phase 3. No public profile URL in Phase 0.

7. User flow — sequence (happy path)



On budget kill: worker sets `Run.status = killed_budget`, SSE event explains tokens/EUR cap, no silent timeout.

8. API surface and events

All routes prefixed `/api`. Auth: Bearer access JWT (15 min) + httpOnly refresh cookie. Role guard on every route except catalogue read of published briefs (optional public) and auth.

Method	Path	Auth	Notes
POST	<code>/auth/magic-link</code>	no	email → mailed token
POST	<code>/auth/magic-link/consume</code>	no	sets cookies
POST	<code>/auth/oauth/:provider</code>	no	Google / GitHub
POST	<code>/auth/refresh</code>	cookie	rotate refresh
GET	<code>/exercises</code>	yes	filter simulator, difficulty, skill
GET	<code>/exercises/:slug</code>	yes	brief, schema, public sample, budgets — never hidden
POST	<code>/attempts</code>	yes	start attempt
POST	<code>/attempts/:id/submissions</code>	yes	enqueue run
GET	<code>/runs/:id</code>	yes	poll status
GET	<code>/runs/:id/stream</code>	yes	SSE
GET	<code>/runs/:id/grade</code>	yes	scorecard, rotated cases, failure classes
GET	<code>/runs/:id/trace</code>	yes	instrumentation
POST	<code>/exercises/:slug/hints/next</code>	yes	unlock
GET	<code>/me/progress</code>	yes	attempts, solves, radar
POST	<code>/simulations/g1/turns</code>	yes	interactive concierge
POST	<code>/internal/ingest/exercises</code>	signed	GitHub Action; metadata only
GET	<code>/internal/cost</code>	admin	cost dashboard

8.1 SSE events

queued, started, step (validate|execute|measure|judge|score), metric (partial), done, inconclusive, killed_budget, error.

8.2 Ownership

Runs, grades, traces are readable only by the owning user (or admin). Spec §9.3: they are durable and addressable so a later share token can be added without moving blobs. Phase 0 does not implement share links.

9. Security, budgets, anti-cheat

9.1 Execution safety (Phase 0)

- No Docker sandbox. No learner Python. No `eval()`.
- RE2 only for DSL patterns. Assertion timeouts as specified.
- Slice Spec: comparison + boolean over published `meta.*` fields.
- Primary live threat: prompt injection of judges via R3/E2/G3 learner text. Mitigate with quarantine delimiters, structured verdict schema, evidence binding, ~40-payload adversarial CI suite.

9.2 Budget kill

Gateway wraps every model call. Counters on the Run: tokens in/out, call count, `costEurMicros`. Crossing any cap in `Exercise.budget` sets `killed_budget` and stops the job. Wall-clock is a worker safety timeout, not a learner-facing gate.

Exercise	EUR cap / attempt
R1 / E1 / E3	€0.02 / €0.001 / €0.001
R2 / R4	€0.30
R3	€0.35 (highest)
E2	€0.25 (240 learner-judge calls)
G1 / G2 / G3	€0.05 / €0.08 / €0.20

Blended mean target $\leq$ €0.20. O8 (unit economics vs Pro price) is a Phase 1 pricing call, not a Phase 0 schema change.

9.3 Anti-cheat

- Hidden `jsonl` never in `public` schema, never in API JSON, never in error messages.
- Rotated failing cases: R1 = 3 of 30; R4 = 5 improved + 3 degraded; E1 = 3 FP + 3 FN; G3 = errors from the 100-item sample only.
- `sampleSeed` on Run makes G3 replayable for support without publishing the pool.
- Accept that a determined cheater beats a practice platform. Do not cripple UX.

9.4 Performance budgets

Catalogue/workspace < 1.5 s p95. Job start < 3 s. Grade < 60 s p95. Progress via SSE.

10. Phase 0 delivery sequence

Week	Engineering milestone
0	Lock O1 (model provider). Thresholds cannot be tuned without it.
1	Prisma schema + two DB roles + migrate. Nest skeleton (API + worker). Model gateway + gen_cache. exercise.yaml ingest Action. Labelling guidelines (content).
2	Go/no-go: RAG harness + R1 end-to-end, $\geq 99\%$ re-grade stability. Vertical slice through workspace.
3	Workspace + trace against R1. Start authoring R3 and G3 (hardest content first).
4	R2, R4. Judge service with quarantine + structured verdicts. Double-annotation underway.
5	Eval harness + frozen fixtures. Freeze Assertion DSL grammar. E1, E2.
6	Guardrail harness, shared normaliser, tool-call log, G2 policy checker. G1, G2. E3 + Slice Spec.
7	G3 pool-and-sample. Budget kill tests. Leakage tests. Adversarial grader suite. RE2 DoS test.
8	Threshold tune, 20 external testers, feedback iteration, acceptance sign-off.

If R1 is not stable by end of week 2, the schedule is wrong. If R3/G3 authoring overruns at week 3, cut to 8 exercises (drop R2 and E3) — do not cut the tester round.

10.1 What this replaces in the current repo

Today	Phase 0 target
Express + Mongoose + MongoDB	NestJS + Prisma + PostgreSQL
In-process simulation engines	BullMQ worker + harness folders
Course / LMS modules	Out of scope — do not port
Code-exec / terminal sandboxes	Phase 1, not now

10.2 Open decisions that still block week 1

- **O1** Model provider for graders/judges.
- **O4** Synthetic corpus (recommended) vs licensed docs.
- **O7** Hands-on Agent Breaker scoring check before any external comparison claim.